Pod Lead Validation Guide

> **Version:** 1.0
> **Target Migration:** DBR 13.3 LTS → 17.3 LTS
> **Last Updated:** February 2026

Quick Navigation

Jump directly to the relevant section:

Validation Area	Guide Section	Quick Link
Getting Started	Workflow identification and categorization	Section 1
Code Migration	How to verify migration complete	Section 2
Row Count	How to compare row counts	Section 3.1
Business Metrics (P1)	How to validate business metrics	Section 3.2
Downstream Reports (P1)	How to validate downstream systems	Section 4
At-Scale UAT (P1)	How to perform at-scale UAT	Section 5
Performance	How to check performance	Section 6
P2 Workflows	Simplified P2 process	Section 7
Final Sign-Off	Completing approvals	Section 8
SQL Templates	Row count & metrics queries	Appendix A
Troubleshooting	Common issues	Appendix B

How to Use This Guide

This is a self-contained guide covering the full validation process. It provides:

- ✅ **Step-by-step instructions** for each validation type
- ✅ **Acceptance criteria** and thresholds
- ✅ **SQL templates** for data comparisons (Appendix A)
- ✅ **Troubleshooting guidance** for common issues (Appendix B)
- ✅ **Escalation procedures** for performance regressions

Work through each section in order, confirming each validation step as you go.

Section 1: Pre-Validation Setup

Identifying Your Workflows

Step 1: Gather the full list of workflows assigned to your POD.

Step 2: Count and categorize your workflows:

- **P1 (Critical):** Production-critical, revenue-impacting, regulatory, or customer-facing
- **P2 (Standard):** Supporting workflows with lower business impact
- **Serverless:** Exclude these (already on compatible runtime)

Step 3: Record the counts as your workflow summary before proceeding.

Section 2: Code Migration Validation

What to Verify

This check confirms that code migration is complete and the workflow executes successfully on DBR 17.3.

Verification Steps

Step 1: Confirm Developer Completion

- Ask the developer: "Has the code migration been completed for [workflow name]?"
- Verify they have applied all breaking change fixes identified by the migration agent or profiler

Step 2: Verify Successful Execution

- Check that the workflow has been run at least once on DBR 17.3 LTS cluster
- Confirm the run completed without errors
- Review execution logs for any critical warnings or deprecations

Step 3: Validate Configuration

- Confirm job cluster configuration points to DBR 17.3 LTS
- Verify all library dependencies are compatible (no version conflicts)

Step 4: Peer Review

- Ensure a second team member has reviewed the code changes
- This provides quality assurance and knowledge sharing

Code Migration Completion Criteria

Code migration is complete when you can confirm:

- ✅ Code migration is complete
- ✅ Workflow runs successfully on DBR 17.3
- ✅ No blocking errors or warnings
- ✅ Peer review completed

Common Issues

Issue	Resolution

Issue	Resolution
Job fails on DBR 17.3	Log the issue, work with developer to fix
Cluster config still on 13.3	Update job definition to use DBR 17.3 cluster
Library compatibility errors	Update library versions or find compatible alternatives
Deprecation warnings in logs	Document for future cleanup (not blocking)

Section 3: Data Quality Validation

3.1 Row Count Validation (All Priorities)

This check confirms that output data volumes match exactly between DBR versions.

What to Compare

Run the workflow on both DBR 13.3 (baseline) and DBR 17.3 (target), then compare:

- Total row count in output table(s)
- Unique key count (if applicable)
- Row counts by partition (for partitioned tables)

How to Capture Row Counts

Option 1: Simple Query

```
SELECT COUNT(*) as row_count FROM {output_table};
```

Option 2: Detailed Comparison (Recommended) See Appendix A.1 for comprehensive SQL template that compares:

- Total rows
- Unique keys
- Null counts
- Partition counts

Acceptance Criteria

Criteria	Requirement	Action if Failed
Total Row Count	**Exact match (0% variance)**	Investigate immediately - could indicate data loss/duplication
Unique Keys	**Exact match**	Check for duplicate key issues
Schema	**Same columns, same types**	Document any intentional changes

Common Causes of Mismatches

Issue	Likely Cause	Investigation
Fewer rows in 17.3	Filtering logic changed, null handling different	Review WHERE clauses, check null behavior changes
More rows in 17.3	Deduplication logic changed, cartesian join	Review JOIN conditions, check distinct/groupby logic
Different unique keys	Key generation logic affected	Check UDF behavior, hash functions, random number generation

3.2 Business Metrics Validation (P1 Only)

This check confirms that critical business calculations produce consistent results.

What Metrics to Validate

For P1 workflows, identify and compare key business metrics:

Metric Type	Examples	Why It Matters
Financial Totals	Revenue, Cost, Profit, Balance	Affects financial reporting and regulatory compliance
Key Counts	Customer count, Transaction count, Order count	Used for business KPIs and forecasting
Calculated Averages	Average order value, Mean processing time	Impacts operational decisions
Percentages/Rates	Conversion rate, Error rate, Success rate	Used in dashboards and SLA monitoring

How to Compare Metrics

Step 1: Work with the developer or business stakeholder to identify the 3-5 most critical metrics for each workflow

Step 2: Capture these metrics from both DBR versions using comparison queries (see Appendix A.2)

Step 3: Calculate variance and assess acceptability

Acceptance Criteria

Metric Type	Acceptable Variance	Rationale
Financial Totals	**Exact match required**	Any difference could impact financial reporting
Counts	**Exact match required**	Count differences indicate data issues

Metric Type	Acceptable Variance	Rationale
Averages	**≤0.01% variance**	Minor floating-point differences acceptable
Percentages	**≤0.01% variance**	Minor rounding differences acceptable

Business Stakeholder Approval

- P1 business metrics **must** be reviewed and approved by the business stakeholder
- Document their approval alongside the validation record
- Save evidence (screenshots, comparison reports) for audit purposes

---

3.3 Schema Validation (Implicit in Row Count Check)

Schema validation is typically performed as part of row count validation. Confirm:

- Output table has the same columns
- Data types are consistent
- No unexpected new columns or missing columns

If schema differences exist, document whether they are:

- ✅ **Intentional** (e.g., new column added as part of migration improvements)
- ❌ **Unintentional** (requires investigation and fix)

Section 4: Downstream Report Validation (P1 Only)

Why This Matters (P1 Only)

P1 workflows often feed critical dashboards, reports, and downstream systems. Even if the workflow runs successfully and data looks correct, breaking changes in data format, schema, or values can break these consumers.

What to Validate

For each P1 workflow, identify and validate all downstream consumers:

Consumer Type	Examples	What to Check
BI Dashboards	Tableau, Power BI, Databricks SQL dashboards	Report loads, visualizations render, metrics match
Other Workflows	Downstream ETL jobs that read this output	Jobs run successfully, no schema errors
External APIs	Data exported to external systems	API calls succeed, data format accepted
Data Shares	Delta Sharing to partners	Share accessible, no permission issues

Consumer Type	Examples	What to Check
ML Models	Feature tables consumed by models	Features generate correctly, no null increases

Validation Process

Step 1: Identify Downstream Consumers

- Work with the developer/data engineer to list all downstream consumers
- Ask: "What reads from this table?" or "What depends on this workflow's output?"

Step 2: Notify Owners

- Contact the owner of each downstream system
- Inform them: "We've migrated [workflow] to DBR 17.3, please validate your [dashboard/job] still works correctly"

Step 3: Validate Each Consumer

For **Dashboards/Reports:**

- Open the dashboard/report
- Verify it loads without errors
- Check that key visualizations render correctly
- Spot-check 2-3 key metrics match expected values
- Get owner confirmation

For **Downstream Workflows:**

- Verify the downstream job has run successfully on updated data
- Check for schema-related errors in logs
- Confirm row counts are as expected

For **External Systems:**

- Test the integration end-to-end
- Verify data format is still accepted
- Check error logs for any new issues

Step 4: Document Approval

- Record the validation result and date
- Note the validator name for each downstream consumer

Report Validation Completion Criteria

Report validation is complete when you can confirm:

- ✅ All downstream consumers identified
- ✅ Owners notified and engaged

- ✅ Each consumer validated and working correctly
- ✅ Any issues resolved or documented

Section 5: At-Scale UAT Testing (P1 Only)

Why At-Scale Testing is Required (P1 Only)

P1 workflows are production-critical. Testing with small sample data may not reveal issues that only appear at production scale:

- Performance degradation under load
- Memory/resource constraints
- Edge cases in large datasets
- Concurrency issues
- Data skew problems

At-scale UAT is mandatory for all P1 workflows before production deployment.

UAT Requirements

Requirement	Criteria	Why It Matters
Data Volume	≥80% of production data volume	Reveals scale-dependent issues
Environment	Production-equivalent cluster specs	Ensures realistic resource usage
Execution	Complete end-to-end without errors	Proves workflow stability
Duration	Within acceptable range of prod timing	Confirms no severe performance degradation

How to Perform UAT

Step 1: Prepare UAT Environment

- Verify UAT cluster is configured with DBR 17.3 LTS
- Ensure cluster specs match production (node type, worker count, autoscaling)
- Confirm production-scale test data is available (≥80% volume)

Step 2: Execute Workflow

- Run the workflow end-to-end on test data
- Monitor execution for errors, warnings, or unusual behavior
- Record start time, end time, and total duration
- Calculate data volume processed as % of production

Step 3: Validate Results

- Verify workflow completed successfully (no errors)
- Check output data quality (row counts, sample validation)
- Confirm duration is within acceptable range (see Performance section)

- Review resource utilization (memory, CPU, shuffle)

Step 4: Obtain Sign-Off

UAT sign-off requires approval from:

- **Technical Lead:** Confirms technical execution success
- **Business Stakeholder:** Confirms business requirements met
- **QA Representative:** (if applicable) Confirms quality standards met

UAT Completion Criteria

UAT is complete when you can confirm:

- ✅ UAT executed with ≥80% production data volume
- ✅ Workflow completed successfully without errors
- ✅ Performance within acceptable limits
- ✅ Required sign-offs obtained (Tech Lead + Business Stakeholder)
- ✅ UAT evidence documented

UAT Documentation

Record the following for each P1 workflow:

- UAT execution date
- Data volume tested (% of production)
- Test duration
- Success/failure status
- Names of approvers
- Any issues discovered and resolutions

Section 6: Performance Regression Testing

Why Performance Testing Matters

DBR upgrades can impact job performance due to:

- Query optimizer changes
- Spark execution engine improvements or regressions
- Auto Loader behavior changes
- Photon runtime differences
- Shuffle and partitioning strategy changes

Performance testing ensures workflows won't run unacceptably slower on the new runtime.

How to Measure Performance

Step 1: Identify Baseline

- Find recent production runs on DBR 13.3
- Use average duration from last 3-5 runs (not a single run)

- Record duration in minutes

Step 2: Measure Target Performance

- Run the workflow on DBR 17.3 (can use UAT test run)
- Use average of 2-3 runs if possible
- Record duration in minutes

Step 3: Calculate Change

```
Change % = ((DBR 17.3 Duration - DBR 13.3 Duration) / DBR 13.3 Duration) ×
100
```

Example:

- DBR 13.3: 45 minutes
- DBR 17.3: 52 minutes
- Change: ((52 - 45) / 45) × 100 = **15.6% slower**

Step 4: Determine Status

Status	Threshold	Action Required
OK	≤10% slower or faster	Document and proceed
WARN	10-25% slower	Requires Tech Lead review
CONCERN	25-50% slower	Requires root cause analysis
BLOCK	>50% slower	Must resolve before deployment

What Each Status Means

✅ OK (≤10% slower)

Action: Document and proceed

- This is within acceptable variance for a major runtime upgrade
- Fill in performance values and mark "OK"
- No additional approval needed

⚠ WARN (10-25% slower)

Action: Document justification and proceed with caution

- Requires documentation of why the slowdown is acceptable
- May be due to known Spark 4.0 behavior changes
- Get Tech Lead review and approval
- Document the justification alongside the validation record

Example justification:

> "15% slower due to Spark 4.0 query optimizer changes. Acceptable tradeoff for long-term runtime support and new features."

🔶 **CONCERN (25-50% slower)**

Action: Root cause analysis required

- Escalate to Tech Lead + Platform Team
- Investigate specific cause (see Investigation Guide below)
- May require configuration tuning or code optimization
- Document investigation findings and mitigation plan
- Requires Platform Team approval to proceed

🛑 **BLOCK (>50% slower)**

Action: Must resolve before deployment

- Unacceptable performance regression
- Requires Platform Team investigation
- May indicate a bug or misconfiguration
- Cannot proceed to production until resolved
- Platform Team approval required after resolution

Performance Investigation Guide

If a workflow shows WARN, CONCERN, or BLOCK status, investigate these common causes:

Cause	How to Check	Potential Fix
Query plan changed	Compare `EXPLAIN` output between versions	Add optimizer hints, adjust query structure
Shuffle increased	Check Spark UI shuffle read/write sizes	Tune `spark.sql.shuffle.partitions`
Cache not used	Review Spark UI storage tab	Explicitly cache hot DataFrames
Auto Loader slower	Check file discovery time	Set `cloudFiles.useIncrementalListing=auto`
Photon disabled	Check cluster config	Ensure Photon is enabled if available
Data skew	Check partition size distribution	Repartition or add salting

For investigation assistance, contact your Platform Team.

Required Approvals by Status

Status	Who Must Approve	When to Get Approval

Status	Who Must Approve	When to Get Approval
OK	Pod Lead (you)	As part of normal sign-off
WARN	Pod Lead + Tech Lead	Before final sign-off
CONCERN	Pod Lead + Tech Lead + Platform Team	Before final sign-off
BLOCK	Must resolve issue first	After resolution, Platform Team approves

Performance Record Format

For each workflow, record:

- DBR 13.3 duration (in minutes)
- DBR 17.3 duration (in minutes)
- Change percentage
- Status (OK/WARN/CONCERN/BLOCK)

Example: `45min → 52min (+15.6%)` — Status: `WARN`

Section 7: P2 Workflow Validation (Simplified)

P2 Validation Requirements

P2 workflows have simpler validation requirements:

What to Check	Required?	Notes
Code Migration	✅ Required	Same as P1 - code complete, runs successfully
Row Count Match	✅ Required	Same as P1 - exact match (0% variance)
Business Metrics	⚪ Optional	Only if workflow has critical metrics
Downstream Reports	❌ Not Required	Skip this check for P2
At-Scale UAT	❌ Not Required	Skip this check for P2
Performance	⚪ Optional	Recommended for long-running jobs (>30 min)

Validation Process for P2

Step 1: Code Migration ✓

- Confirm developer completed migration
- Verify workflow runs successfully on DBR 17.3
- Same as P1 validation

Step 2: Row Count ✓

- Compare output row counts between DBR 13.3 and 17.3
- Must match exactly (0% variance)

- Same as P1 validation

Step 3: Performance (Optional)

- Recommended for workflows that run >30 minutes
- Follow the same process and thresholds as P1 (see Section 6)
- Can skip for quick-running jobs (<30 min)

Step 4: Sign-Off

- Pod Lead sign-off is sufficient for P2
- No Tech Lead or Business Stakeholder required (unless performance is CONCERN/BLOCK)

Why P2 is Simplified

P2 workflows are lower priority and have:

- Less business impact if issues occur
- Easier rollback options
- Lower data volumes typically
- Fewer downstream dependencies

The simplified validation balances risk with effort.

Section 8: Final Sign-Off Process

Sign-Off Authority

Different validations require different levels of approval:

Validation	P1 Approvals Required	P2 Approvals Required
Code Migration	Pod Lead	Pod Lead
Row Count	Pod Lead	Pod Lead
Business Metrics	Pod Lead + Business Stakeholder	N/A
Downstream Reports	Pod Lead + Report Owners	N/A
UAT	Pod Lead + Tech Lead + Business Stakeholder	N/A
Performance (OK)	Pod Lead	Pod Lead
Performance (WARN)	Pod Lead + Tech Lead	Pod Lead + Tech Lead (if checked)
Performance (CONCERN/BLOCK)	Pod Lead + Tech Lead + Platform Team	Pod Lead + Tech Lead + Platform Team

Final Sign-Off Checklist

P1 Workflows

Before signing off, confirm all of the following:

- ✅ All P1 workflows have checkmarks in all required columns
- ✅ Row count validations show 0% variance
- ✅ Business metrics validated with ≤0.01% variance
- ✅ Downstream reports verified
- ✅ UAT completed with ≥80% production data
- ✅ Performance status is OK, or WARN/CONCERN has been approved
- ✅ No BLOCK status workflows (all must be resolved)
- ✅ Business stakeholder reviewed and approved

P2 Workflows

Confirm:

- ✅ All P2 workflows have checkmarks in required columns (Code, Row Count)
- ✅ Row count validations show 0% variance
- ✅ Performance checked if applicable

Issues Verification

- ✅ All blocking issues resolved
- ✅ Non-blocking issues documented
- ✅ All issues and their resolutions recorded

Obtaining Signatures

For P1 Workflows:

1. **Pod Lead** (you): Sign after completing all validation
2. **Tech Lead**: Sign to confirm technical validation completeness
3. **Business Stakeholder**: Sign to confirm business metrics and UAT approval

For P2 Workflows:

1. **Pod Lead** (you): Sign after completing all validation

Post Sign-Off Actions

After all signatures are collected:

1. Update Records

- ☐ Record completion date for each workflow
- ☐ Upload evidence (screenshots, comparison results, UAT documentation)
- ☐ Close any resolved issues

2. Notify BAU/DevOps Team

Send a confirmation to your BAU/DevOps team and CC Platform Engineering and Tech Leads. Include the following summary:

- Total workflows validated (P1 and P2 counts)
- Code migration status (% complete)
- Row count validation results
- Business metrics validation results (P1) with approver name
- Downstream report verification results (P1)
- At-scale UAT results (P1)
- Performance summary (count by OK/WARN/CONCERN status)
- Number of open issues (and whether any are blocking)
- Confirmation that you are ready for production deployment scheduling

3. Archive Documentation

- ☐ Save completed validation records to shared location
- ☐ Archive all validation evidence
- ☐ Keep for audit purposes

What Happens Next

After your sign-off is received:

1. BAU/DevOps will schedule production deployment
2. Your workflows will be updated to use DBR 17.3 LTS clusters
3. You'll be notified of deployment date/time
4. Monitor workflows after deployment for any issues

Appendix A: Validation SQL Templates

A.1 Comprehensive Row Count Comparison

```
-- Template for detailed row count comparison
WITH baseline AS (
    SELECT
        COUNT(*) as total_rows,
        COUNT(DISTINCT {primary_key}) as unique_keys,
        SUM(CASE WHEN {primary_key} IS NULL THEN 1 ELSE 0 END) as
null_keys,
        COUNT(DISTINCT {partition_column}) as partitions
    FROM {schema}.{table}_baseline
),
target AS (
    SELECT
        COUNT(*) as total_rows,
        COUNT(DISTINCT {primary_key}) as unique_keys,
        SUM(CASE WHEN {primary_key} IS NULL THEN 1 ELSE 0 END) as
null_keys,
        COUNT(DISTINCT {partition_column}) as partitions
```

```sql
    FROM {schema}.{table}_target
)
SELECT
    'Baseline (13.3)' as version,
    b.total_rows,
    b.unique_keys,
    b.null_keys,
    b.partitions
FROM baseline b
UNION ALL
SELECT
    'Target (17.3)' as version,
    t.total_rows,
    t.unique_keys,
    t.null_keys,
    t.partitions
FROM target t;
```

A.2 Business Metrics Comparison (P1)

```sql
-- Template for comparing key business metrics
SELECT
    '{workflow_name}' as workflow,
    'DBR 13.3' as version,
    SUM(revenue) as total_revenue,
    COUNT(DISTINCT customer_id) as unique_customers,
    COUNT(*) as transaction_count,
    AVG(order_value) as avg_order_value,
    MAX(process_date) as latest_process_date
FROM {schema}.{table}_baseline

UNION ALL

SELECT
    '{workflow_name}' as workflow,
    'DBR 17.3' as version,
    SUM(revenue) as total_revenue,
    COUNT(DISTINCT customer_id) as unique_customers,
    COUNT(*) as transaction_count,
    AVG(order_value) as avg_order_value,
    MAX(process_date) as latest_process_date
FROM {schema}.{table}_target;

-- Calculate variance
WITH baseline AS (
    SELECT SUM(revenue) as revenue FROM {schema}.{table}_baseline
),
target AS (
    SELECT SUM(revenue) as revenue FROM {schema}.{table}_target
)
SELECT
```

```
    b.revenue as baseline_revenue,
    t.revenue as target_revenue,
    t.revenue - b.revenue as absolute_difference,
    CASE
        WHEN b.revenue = 0 THEN NULL
        ELSE ((t.revenue - b.revenue) / b.revenue) * 100
    END as variance_percent
FROM baseline b, target t;
```

A.3 Data Hash Comparison (for exact data validation)

```
-- Compare data using hash for exact match verification
WITH baseline_hash AS (
    SELECT
        {primary_key},
        hash(concat_ws('|', *)) as row_hash
    FROM {schema}.{table}_baseline
),
target_hash AS (
    SELECT
        {primary_key},
        hash(concat_ws('|', *)) as row_hash
    FROM {schema}.{table}_target
)
SELECT
    COALESCE(b.{primary_key}, t.{primary_key}) as key,
    CASE
        WHEN b.row_hash IS NULL THEN 'Missing in baseline'
        WHEN t.row_hash IS NULL THEN 'Missing in target'
        WHEN b.row_hash != t.row_hash THEN 'Hash mismatch'
        ELSE 'Match'
    END as comparison_result
FROM baseline_hash b
FULL OUTER JOIN target_hash t ON b.{primary_key} = t.{primary_key}
WHERE b.row_hash IS NULL
   OR t.row_hash IS NULL
   OR b.row_hash != t.row_hash;
```

Appendix B: Common Issues and Resolutions

Issue	Symptom	Resolution
Row count mismatch	Target has fewer/more rows	Check for filter changes, null handling, deduplication logic
Business metric variance	Aggregations differ slightly	Check for floating-point precision, rounding changes

Issue	Symptom	Resolution
Schema differences	New/missing columns	Verify schema evolution settings, check for dynamic columns
Performance regression	>25% slower	Review query plans, check shuffle settings, verify Photon config
Downstream report errors	Dashboard shows errors	Check for type changes, null handling, column renames

Document History

Version	Date	Author	Changes
1.0	February 2026	Platform Team	Initial version